



THE UNIVERSITY OF
SYDNEY

Room Number _____

Seat Number _____

Student Number

Practice Exam II

MOCK EXAM PAPER

(This is not a real exam and not worth any marks)

This is practice to see the layout and example questions for your final exam

MOCK EXAMINATION Practice Exam II

~~Semester 1 – Mock 2025~~ **Practice Exam II**

COMP9001 Introduction to Programming

EXAM WRITING TIME: 2 hours

READING TIME: 10 minutes

EXAM CONDITIONS:

This is a RESTRICTED OPEN book exam - specified materials permitted

MATERIALS PERMITTED IN THE EXAM VENUE:

(No electronic aids are permitted e.g. laptops, phones)

Calculator – non-programmable

Formula sheet (provided in the exam paper by unit coordinator)

One A4 sheet of handwritten notes double-sided

MATERIALS TO BE SUPPLIED TO STUDENTS:

Blank scratch paper (3 sheets)

INSTRUCTIONS TO STUDENTS:

Do not turn over this page until you are told!

- This paper contains nine (9) questions with a total of 100 marks. Answer ALL questions.
 - Questions 1–4 (**Essentials** Section): 40 marks
 - Questions 5–9 (**Advanced** Section): 60 marks
- Please read all the questions first before you begin writing.
- Answer all questions in the spaces provided in this question booklet. Do not write your answers in the answer booklets, which are only to be used for drafting and will not be graded.
- Return this question booklet and any additional answer booklets or materials.
- Write your final answers in black or blue ink, not pencil. Take care to write clearly and legibly. Drawing diagrams using pencil is permitted.
- Please write your student number in the space provided at the top of every odd numbered page of this question booklet.
- State clearly in spaces below the question if the answer is continued on the extra working pages on pages 25-28 of this question booklet.
- Do not remove any tear or remove any pages from the booklet. Staff are not responsible for missing pages in this case. If you find there are missing pages, please alert the examination supervisors.
- You may refer to the formula sheet for any relevant reference material.

_____ This page was left blank intentionally. Do not write on it. _____

Question 1 (18 marks):

/18

1.a:

/4

Mark the following statements true or false:

- (a) ☐ T The keyword `def` is used to define a function in Python.
- (b) ☐ F The expression `5 // 2` will return `2.5` as a float.
- (c) ☐ T Dictionaries in Python store data in key-value pairs.
- (d) ☐ F The statement `if x = 5:` is valid syntax for an if statement.
- (e) ☐ T The expression `len("hello")` returns the number 5.
- (f) ☐ F In Python, you must declare a variable's type before using it.
- (g) ☐ F You can use the `append()` method to add an item to a Python tuple.
- (h) ☐ T You can use a `for` loop to iterate over the characters in a string.

1.b:

/1

What is the value of `num` after `num = 5%3//5` is executed?

Put your answer here:

0

1.c:

/1

What is the value of `result` after the following code is executed?

```
1 x = (8 * 2) <= 15
2 y = (10 % 3) == 1
3 result = (x and not y) or (y and not x)
```

Put your answer here:

True

1.d:

/1

Create a variable named `letters` and assign it a list containing the four strings `The`, `test`, `world`, and `hello`.

```
letters = ['The', 'test', 'world', 'hello']
```

1.e:

/1

What is the output after the following code is executed?

```
1 i = 3
2 while i < 15:
3     i += 1
4     if i == 10:
5         print(i)
6         break
```

10

1.f:

/1

Given a user input `id_num` (an integer), print the ID number as a 4-digit zero-padded string.

For example:

- If the input is 7, the output should be: ID: 0007
- If the input is 42, the output should be: ID: 0042
- If the input is 1234, the output should be: ID: 1234

Complete the following program:

```
id_num = int(input())
```

```
print(f'ID: {id_num:04}')
```

1.g:

/1

Given a string `line`, make a new string `new_line` that repeats `line` 5 times.

Note: We will not accept answers that doesn't use `line` in its expression.

Complete the following program:

```
line = 'COMP9001'
```

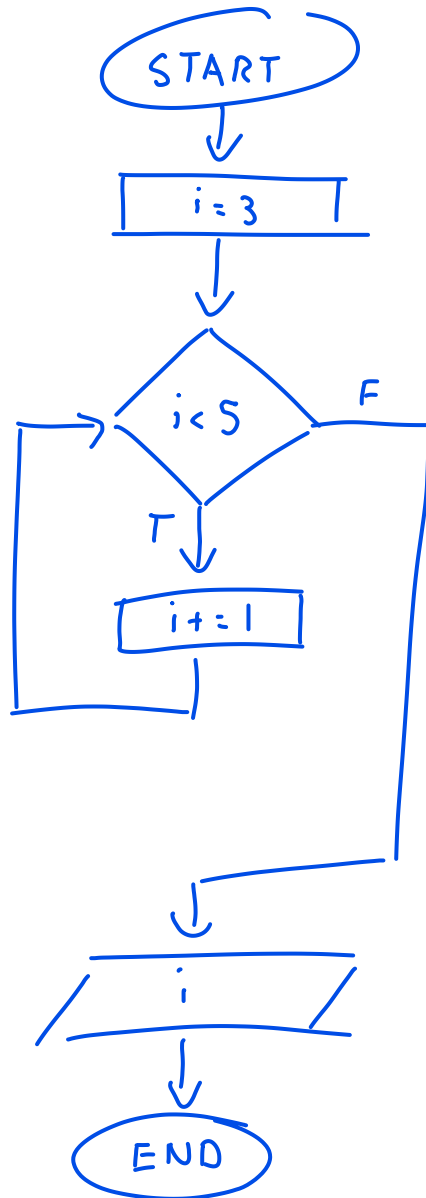
```
new_line = line * 5 # string concatenation
```

1.h:

/4

Draw a flowchart that represents the control flow of the following code:

```
1 i = 3
2 while i < 5:
3     i += 1
4 print(i)
```



1.i:

/2

Given the following code:

```
1 values = [3, 6, 2, 5]
2 total = 0
3
4 for val in values:
5     if val > 4:
6         total = total + 2
7     else:
8         total = total + 1
```

How many times will the loop iterate?

4

What will be the final value of `total` after the loop is complete?

6

1.j:

/2

Given the following program:

```
1 def bar(ls, threshold):
2     for i in range(len(ls)):
3         if ls[i] < threshold:
4             ls[i] = ls[i] * 2
5
6 data = [3, 5, 7, 2]
7 bar(data, 6)
```

(i) What is the value of `data` after `bar(data, 6)` is executed? (1)

[6, 10, 7, 4]

(ii) Lists in Python are passed to functions by reference. (1)
Fill in the blank with one of: value or reference

Question 2 (6 marks):

/6

the function `find_sum` is supposed to return the sum of even numbers in a list as follows:

```
ls = [1,2,3,4,5]

print(find_sum(ls)) # should print 15
```

Below is the entire code for `find_sum`. Find as many *different* errors (of any kind, syntax, or logical) as you can.

Note: A single line may contain more than one error.

- For each error you find, state the line number and briefly explain why it is an error (simply listing the line number without an explanation will receive no marks).

```
1 def find_sum(ls)
2     for i in range(len(ls))
3         if i % 2 == 1:
4             ls_sum += i
5         return ls
```

You need to identify 6 errors for full marks.

- (a) line: ; error is:
- (b) line: ; error is:
- (c) line: ; error is:
- (d) line: ; error is:
- (e) line: ; error is:
- (f) line: ; error is:
- (g) line: ; error is:
- (h) line: ; error is:

Question 3 (10 marks):

/10

3.a:

/4

Write a function `has_double_letter(word)` that:

- Takes one parameter:
 - `word`: a non-empty string.
- Returns `True` if the word contains two identical characters in a row, otherwise returns `False`.

Example usages:

- Calling `has_double_letter('apple')` returns `True` (because of `pp`).
- Calling `has_double_letter('banana')` returns `False`.
- Calling `has_double_letter('coffee')` returns `True` (because of `ff` and `ee`).

Restrictions: Do **not** use the `in` operator to check for substrings. Do **not** use slicing or string methods such as `find`, `count`, or `index`. You may use the `in` keyword **only** in `for` loops (e.g., `for i in range(len(word))`).

```
def has_double_letter(word):  
    for i in range(len(word)-1): #skip last char  
        if word[i] == word[i+1]:  
            return True  
    return False
```

3.b:

/6

Using your `has_double_letter` function from part (3.a), write a program that:

- Repeatedly prompts the user to enter a word, until the user enters 'quit'.
- For each word entered (other than 'quit'):
 - Check if it contains double letters using your function.
 - If it does, add the word to a list called `doubles`.
- After the user enters 'quit', prints the list `doubles`.

Note: Use the existing `has_double_letter` function from part (3.a); no need to rewrite it. If you were unable to implement it, assume it exists.

Example run [user inputs (i.e., words typed by the user) are shown in **bold**]:

```
$ python3 program.py
Word: apple
Word: banana
Word: date
Word: carrot
Word: quit
Final: ['apple', 'carrot']
```

```
final_words = []
while True:
    word = input('word: ')
    if word == 'quit':
        break
    if has_double_letter(word):
        final_words.append(word)

print('Final:', final_words)
```

Question 4 (6 marks):

/6

Given the following two modules:

library.py

```
1 class Library:
2     def __init__(self, name: str, num_books: int):
3         self.name = name
4         self.num_books = num_books
5
6     def borrow_books(self, amount: int):
7         self.num_books -= amount
8
9     def restock(self, amount: int):
10        self.num_books += amount
```

library_program.py

```
1 from library import Library
2
3 city_library = Library("City Library", 5)
4 uni_library = Library("University Library", 10)
5
6 city_library.borrow_books(3)
7 uni_library.borrow_books(15)
8 uni_library.restock(5)
9
10 print(city_library.num_books)
11 print(uni_library.num_books)
```

4.a:

/2

What will be the output produced from lines 10 and 11 in `library_program.py` after the program is executed?

Line 10 will output:

2

Line 11 will output:

0

4.b:

/2

Rewrite the `borrow_books` method so that:

- If the amount is not an integer, it prints `Amount must be an integer!`.
- If the amount is greater than the number of books, it prints `Not enough books to borrow!` and it does nothing.
- Otherwise, it subtracts the number of books as normal.

```
def borrow_books(self, amount: int):
```

```
    if type(amount) != int: # can use isinstance() too
        print('Amount must be an integer!')
    elif amount > self.num_books:
        print('Not enough books to borrow!')
    else:
        self.num_books -= amount (deduct)
```

4.c:

/2

State one similarity and difference between a static method and an instance method.

Similarity: both defined in a class.

Difference: instance method relies on an instance to be called, static method does not.

Question 5 (11 marks):

/11

5.a:

/3

The program below does not execute successfully.

```
1 counter = 0
2
3 def login():
4     counter += 1
5     print("User logged in", counter, "time(s).")
6
7 login()
```

When it is run, it produces the error shown.

```
UnboundLocalError: local variable 'counter' referenced before assignment
```

Answer the following:

- (i) Why does the program fail to execute? (1)

counter needs to be initialised before
performing: counter += 1

- (ii) Fix the program by indicating the line number(s) you are changing and writing your updated code (you can add, remove or modify lines as needed). (2)

between
line 3
and line 4: global counter

5.b:

/8

Write a program called `prices_total.py` that takes n strings from the command-line arguments (excluding the filename). Each string should be in the format: `<item_name>:<price>`
The program should:

- Extract and add up all valid prices (floats or integers) from the input and display the total correct to 2 decimal places
- Ignore any string that:
 - Does not contain exactly **one** colon (:
 - Does not have a number as the price part

Example run [user inputs (i.e., words typed by the user) are shown in **bold**]:

```
$ python3 prices_total.py apple:2.5 banana:abc orange:3 mango:1.5
Total: 7.00
```

```
$ python3 prices_total.py chair:29.99 table:89.50 shelf:49.5 invalid_format
Total: 168.99
```

Hint: Use `sys` module, string method `split`, exception handling (`try` and `except`). Refer to the formula sheet if you need more information on these.

```
import sys
total = 0
for arg in sys.argv[1:]: # exclude program name
    tokens = arg.split(':')
    if len(tokens) != 2:
        continue

    try:
        total += float(tokens[1])
    except ValueError:
        pass # do nothing

print(f'Total: {total:.2f}')
```

Continue your answer here

Question 6 (13 marks):

/13

6.a:

/3

Mark the following statements true or false:

- (a) The built-in `ValueError` is typically raised when a function receives an argument of the correct type but inappropriate value.
- (b) In Python, syntax errors can be caught using `try` and `except`.
- (c) You can have multiple `except` blocks after a `try` block.
- (d) If a file does not exist, opening it in `'a'` mode will create it.
- (e) Static methods can access instance attributes directly using `self`.
- (f) if `__eq__` is not defined in a class, comparing objects with `==` will raise an error.

6.b:

/10

Write a program called `count_all_lines.py` that performs the following:

- Opens the file specified from the first command-line argument (excluding the program name). The program should not crash if this argument is missing.
- Counts the total number of words in the file. A word is any sequence of characters separated by whitespace.
- Prints the total word count.
- Ignores any additional command-line arguments.
- If any error occurs, print 'error' and terminate the program.

For example, to run the program with the file `hello.txt`:

```
$ python3 count_all_lines.py hello.txt
```

Make sure you handle and report *any possible exceptions* that might arise. Refer to the last page of the formula sheet if you need a reminder on file methods.

```
import sys

if len(sys.argv) < 2:
    print('error')
else:
    try:
        f = open(sys.argv[1], 'r')
        total_words = 0
        while True:
            line = f.readline()
            if line == '': # end of file
                break
            total_words += len(line.split())
        f.close()
        print(total_words)
    except FileNotFoundError:
        print('error')

print(total_words)  should only print if it succeeded
```

Question 7 (6 marks):

/6

Write a class called `Wizard` that models a wizard in a magical realm.

- Each wizard has a name (string), a power level (integer), and a mana amount (defaults to 50).
- The `__init__` method takes 2 parameters: name (string), power (int) This method sets the wizard's name and power, and sets their mana to the default value.
- A static method `recommended_mana` that takes 1 parameter: power (int). This method returns a recommended starting mana amount, calculated as $\text{power} * 10$.
- A method `cast_spell` that takes 1 parameter: amount (int). This method reduces the wizard's mana by the given amount. If the amount is greater than the current mana, the spell fails and mana remains unchanged.
- A method `train` that takes 1 parameter: amount (int). This method increases the wizard's power by the given amount.
- A method `__str__(self)` that returns a string in the format: "<name> (<power>, <mana>)" where <name>, <power> and <mana> are attributes of the wizard.

```
class Wizard:
    def __init__(self, name, power):
        self.name = name
        self.power = power
        self.mana = 50

    def recommended_mana(power):
        return power * 10

    def cast_spell(self, amount):
        if amount < self.mana:
            self.mana -= amount

    def train(self, amount):
        self.power += amount

    def __str__(self):
        return f'{self.name} ({self.power}, {self.mana})'
```

Question 8 (20 marks):

/20

8.a:

/12

Write a function `balanced_index(nums)` that:

- Takes one parameter:
 - `nums`: a list of integers with at least **3 elements**.
- Returns the index `i` (where $1 \leq i \leq \text{len}(\text{nums}) - 2$) such that the **sum of all elements to the left of `i`** is equal to the **sum of all elements to the right of `i`** (`i` cannot be either ends of the list).
- If no such index exists, the function should return `-1`.

Example usages:

- Calling `balanced_index([1, 2, 3, 4, 6])` returns `3`.
- Calling `balanced_index([2, 4, 2])` returns `1`.
- Calling `balanced_index([10, 1, 2, 3])` returns `-1`.
- Calling `balanced_index([0, 0, 0, 0])` returns `1`.

Your solution should involve loops (`for`, `while`), conditional statements (`if`, `elif`, `else`) and array-like indexing of the list (e.g., `nums[i]`). You may also use variables to accumulate running totals and compare them at each step.

Restrictions: You may use the `in` keyword **only** in `for` loops (e.g., `for num in nums`, `for i in range(n)`, etc.).

You **must not** use the following:

- Built-in functions like `sum()` with slicing (e.g., `sum(nums[:i])`).
- List slicing (e.g., `nums[i:j]`).
- Any external libraries such as `numpy`.

Using any of the above disallowed features will result in **zero marks** for the question.

Write your answer on the next page

Write your answer here

```
def balanced_index(nums):  
    for i in range(1, len(nums) - 1): # i cannot be  
                                        # start or end  
        left = 0  
        for j in range(i): # 0 to i (exclusive)  
                                [ ... i ... ]  
            left += nums[j]  
        right = 0  
        for j in range(i + 1, len(nums)): # i + 1 to end  
                                            [ ... i ... ]  
            right += nums[j]  
        if left == right:  
            return i  
    return -1
```

8.b:

/8

Write a recursive function `mirror(word)` that returns a new string which is the **reverse** of the given string `word`.

The function should:

- Return the original string if it is empty or has only one character.
- Use recursion to build the reversed string one character at a time.
- Not use loops.

Note: You must implement the reversal using **recursion only**.

Example usages:

- Calling `mirror('hello')` returns `'olleh'`.
- Calling `mirror('a')` returns `'a'`.
- Calling `mirror('racecar')` returns `'racecar'`.

Restrictions:

- You **must not** use loops (`for`, `while`).
- You **must not** use built-in functions like `reversed()` or `' '.join()`.

Using any of the above disallowed features will result in **zero marks** for the question.

Part A: Before writing any code, explain what the **base case** and **recursive case** are for this problem. (3 marks)

- (i) What is the base case for this recursion? (**Hint:** it has been stated above!) (1)

when we reach 1 character,
which we just return this
character.

- (ii) When should we recurse? What smaller subproblem should we recurse on? (2)

we recurse when there's 2 characters
or more. we recurse on a smaller
portion of word (we exclude last
character).

Part B: Now implement the function `mirror(word)`. Your function **must** use recursion. Functions that are not recursive will receive **zero marks**. (5)

```
def mirror(word):  
    if len(word) <= 1:  
        return word  
    return word[-1] + mirror(word[:-1])  
  
# [-1] is last index  
# [:-1] is start to last index (exclusive)
```

Question 9 (10 marks):

/10

You have tasked a student with implementing 2 functions in `foo.py`. The docstrings for each function specify the correct behavior of the function (what the function is meant to do).

Below is the implementation of the functions that the student has provided. Your job is to develop test cases in `foo_test.py` to ensure that the student's implementation follows the requirements specified in the docstrings.

- For each function, develop 4 appropriate, non-overlapping test cases (you can define helper functions to help write your tests). (8)
- At least one of these test cases for each function above must either detect a mistake or something missed in the implementation that does not match the function's docstring. (2)

Hint: You may find it helpful to use `assert` statements, as well as exception handling (`try` and `except`). Refer to the formula sheet if you need more information on these.

foo.py

```
1 def square_all(values: list[int]) -> list[int]:
2     '''
3     Squares each integer in the input list.
4
5     Returns:
6     - A list of squared integers.
7     - Returns None if any item in the list is not an integer.
8     '''
9     result = []
10    for v in values:
11        result.append(v * v)
12    return result
13
14
15 def safe_index(item, data: list) -> int:
16     '''
17     Finds the index of the first occurrence of item in the list.
18
19     Returns:
20     - The index of the item if found.
21     - Returns -1 if item is not in the list.
22     - Raises a TypeError if data is not a list.
23     '''
24    if not isinstance(data, list):
25        return -1
26
27    for i in range(len(data)):
28        if data[i] == item:
29            return i
30    return -1
```

Write your answer on the next page

Write your answer here

```
from foo import square-all, safe-index
# one value (test 1)
assert square-all([2]) == [4], 'test1 failed'

# multiple values (test 2)
assert square-all([1,2]) == [1,4], 'test2 failed'

# non-int values (mismatch of docstring) (test 3)
assert square-all(['a']) == None, 'test3 failed'

# empty list (test 4)
assert square-all([]) == [], 'test4 failed'

# item found (test 5)
assert safe-index(2, [1,2,3]) == 1, 'test5 failed'

# item not found (test 6)
assert safe-index(0, [1]) == -1, 'test6 failed'

# test first occurrence (test 7)
assert safe-index(2, [2,2,2]) == 0, 'test7 failed'

(next page for last test)
```

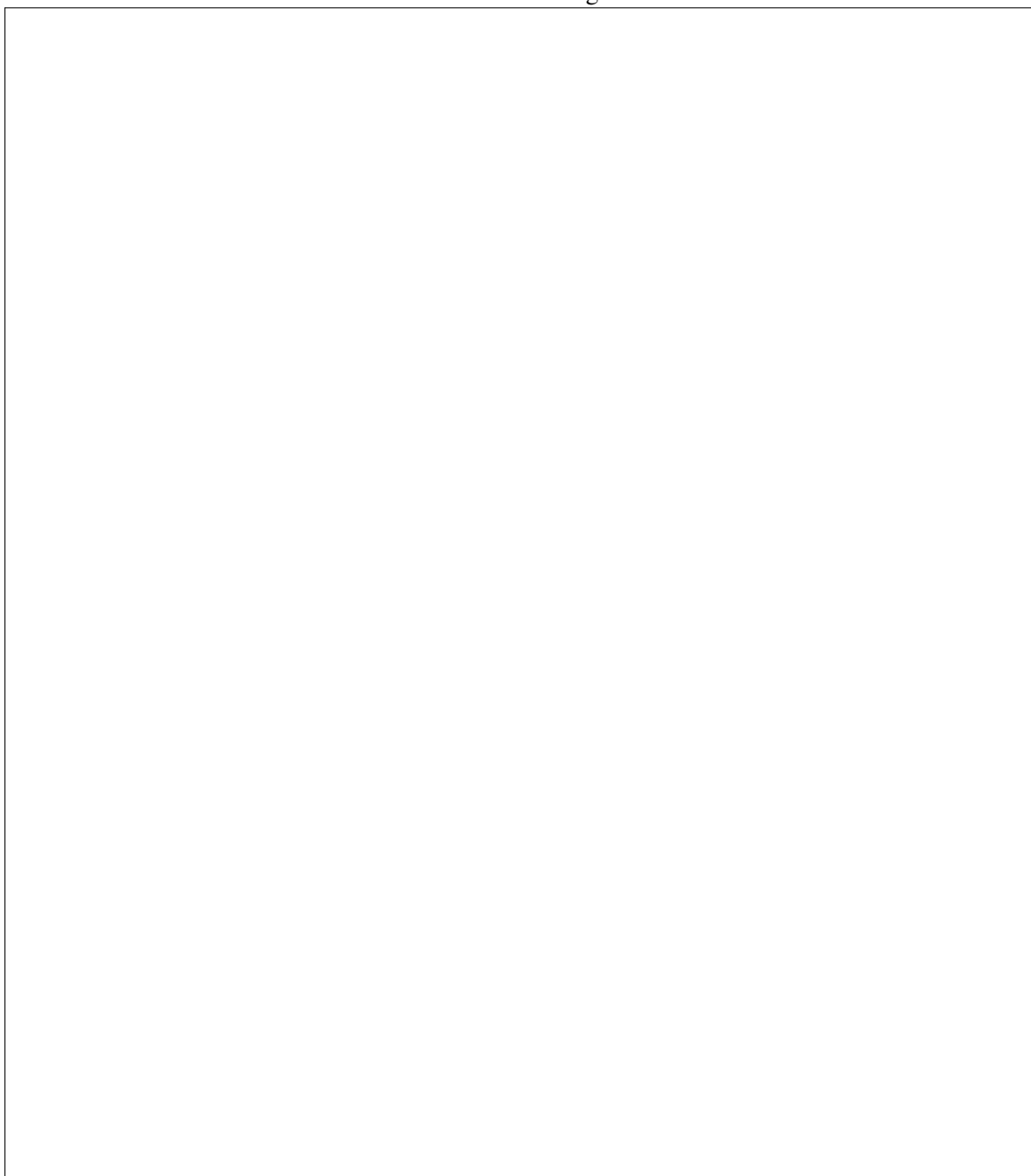
Continue your answer here

```
# test data not list (TypeError) (test 8)
error = False # check if error raised
try:
    safe_index(0, None)
except TypeError:
    error = True
if not error:
    assert False, 'test8 failed'
```

THIS AND FOLLOWING PAGES LEFT INTENTIONALLY BLANK.

Use this and following pages for additional writing space if you run out of space in an answer area. If you want any writing on this page or following pages to be marked, you **MUST** write "see additional pages" in the relevant answer area and clearly indicate the question number you are answering on the relevant page. Any writing outside of this booklet will not be examined.

Extra working



Extra; Write your SID:

Extra working

Formula Sheet

Common types: int, float, bool, str

Common escape characters: \n, \t, \', \", \\

Type information: type(), isinstance(object, class)

Parsing methods: int(), float(), str()

Common collections: [] (list), () (tuple), {} (dictionary)

Collection classes: list, dict, set, tuple

Common collection methods: append, insert, remove, pop, index, reverse, copy

Keywords:

def – Define a function e.g., def foo():

import – Import a module e.g., import sys

raise – Raise an exception e.g., raise Exception()

class – Define a class e.g., class Foo:

assert – Raises AssertionError if condition False e.g., assert x == 0, 'Failed!'

break, continue, return, yield

Common Modules:

sys – Provides access to command-line arguments (sys.argv).

Useful iterable Collection Operations:

sorted() – Returns a sorted list from an iterable.

reversed() – Returns a reverse iterator over an iterable.

list(iterable) – Returns a new list that contains all items from iterable, in the same order.

len() – Returns the length of the iterable.

next() – Returns the next item in the iterator, otherwise raises StopIteration.

iter() – Returns an iterator from the object passed.

Control Flow Syntax:

- if...:... [elif...:...] [else: ...]
- while...: ... [else: ...]
- for... in ...:... [else: ...]
- try: ... except [...as ...]:... [finally: ...]

Common Exceptions: Exception, SyntaxError, ValueError, TypeError, NameError, FileNotFoundError, IndexError, EOFError, KeyError, ZeroDivisionError, AssertionError

-
- **Assignment Operators:** `=` `+=` `*=` `-=` `/=` `//=` `%=`
 - **Arithmetic Operators:** `+` `-` `*` `/` `//` `%` `**`
 - **Comparison Operators** `==` `!=` `>=` `<=` `<` `>`
 - **Logical Operators:** `not` `and` `or`
 - **Membership Operator:** `in`

Input/Output functions:

`print()` – Prints object(s) to standard output using their string representation.

`input()` – Reads a line of input from standard input (keyboard)

`open(path, mode)` – Opens file at path in given mode (read 'r', write 'w', append 'a')

e.g., `my_file = open('a.txt', 'r')`

File methods:

`close()` – Closes the file.

`read(n)` – Reads n characters from the file.

`readline()` – Reads a single line from the file.

`readlines()` – Reads all lines from the file as a list of strings.

`write()` – Writes a string to the file [does not add a newline (`\n`) automatically].

Dictionary - access:

`d[key]` – Returns the value indexed at key in d.

String, list, tuple - access:

`seq[i]` – Returns the element at index i of seq.

Slicing (returns a subsequence of seq):

- `seq[start:end]` – Returns elements from index start (inclusive) to end (exclusive).
- `seq[start:]` – Returns elements from index start to the end of seq.
- `seq[:end]` – Returns elements from the beginning of seq up to index end (exclusive).

String methods (str):

`split(sep)` – Splits the string into a list of substrings using the specified separator sep. If sep is not provided, it defaults to whitespace e.g., `'A B'.split(' ')` returns `['A', 'B']`

`lower()` – Returns a string in lowercase.

`upper()` – Returns a string in uppercase.

`strip(chars)` – Returns a string with the specified chars removed from both the start and end of the string. If chars is not provided, it defaults to whitespace.

`format()` – Returns a formatted string using placeholders in `{}`.